

Динамическое изменение UI

Динамическое изменение UI

- В приложении может возникнуть необходимость добавления новых элементов на форму.
- Рассмотрим простейший пример – добавление новых полей для ввода текста.
- Если максимальное количество полей известно заранее, задачу можно решить с помощью дизайнера – просто создать все поля заранее, а во время выполнения скрывать ненужные поля.

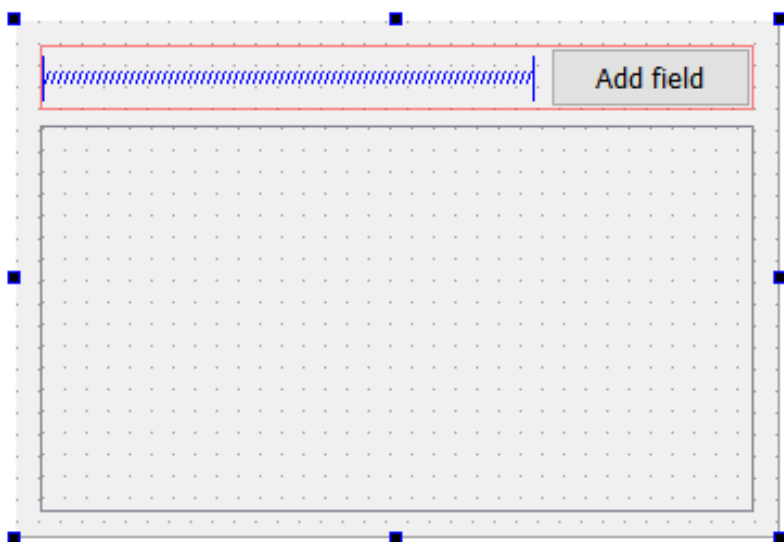
Динамическое изменение UI

- Однако в том случае, когда максимальное количество возможных полей неизвестно, дизайнер не поможет.
- Для решения проблемы необходимо как минимум реализовать следующие методы:
 - добавление поля;
 - удаление поля;
- Помимо этого нужно сделать так, чтобы приложение адекватно реагировало на добавление и удаление полей.

Необходимые классы для решения задачи

- QScrollArea - очень удобный класс, который автоматически добавляет полосы прокрутки, если его содержимое не помещается целиком;
- QHBoxLayout / QVBoxLayout – компоновщики, с помощью которых будет выполняться группировка полей ввода;
- QLabel / QLineEdit – для ввода/вывода текста;
- QPushButton – кнопки – наше всё!

Объект формы



Объект	Класс
▼ DynamicWidget	QWidget
▼ QHBoxLayout	QHBoxLayout
horizontalSpacer	Spacer
pushButton_AddField	QPushButton
▼ scrollArea	QScrollArea
scrollAreaWidgetContents	QWidget

DynamicWidget.h

```
#include <QWidget>
```

```
class QVBoxLayout;
```

```
namespace Ui {  
class DynamicWidget;  
}
```

```
class DynamicWidget : public QWidget  
{  
    Q_OBJECT
```

```
public:
```

```
    explicit DynamicWidget(QWidget *parent = nullptr);
```

```
    ~DynamicWidget();
```

```
    void removeField(const int &index);
```

```
public slots:
```

```
    void addField();
```

```
    void removeField();
```

```
private:
```

```
    QWidget *createField();
```

```
    Ui::DynamicWidget *ui;
```

```
    QVBoxLayout *fieldsLayout_;
```

```
    QVector<QObject*> buttons_;
```

```
};
```

Опережающее описание сообщает компилятору C++ о том, что этот класс существует. Позволяет использовать в заголовочном файле указатели этого класса. Уменьшает время компиляции проекта в сравнении с `#include <QVBoxLayout>`

Определяет индекс и вызывает

Компоновщик-контейнер для полей ввода

Хранение указателей на кнопки позволит найти кнопку, вызвавшую слот

Родительский класс QPushButton

DynamicWidget::createField()

```
QWidget *DynamicWidget::createField()
{
    QWidget *field = new QWidget(this);
    QHBoxLayout *layout = new QHBoxLayout();
    layout->setContentsMargins(0, 0, 0, 0);
    QLabel *label = new QLabel(field);
    layout->addWidget(label);
    layout->setStretchFactor(label, 1);
    QLineEdit *lineEdit = new QLineEdit(field);
    connect(lineEdit, SIGNAL(textChanged(const QString &)), label, SLOT(setText(const QString &)));
    layout->addWidget(lineEdit);
    layout->setStretchFactor(lineEdit, 1);
    QPushButton *button = new QPushButton(field);
    button->setText("Remove field");
    connect(button, SIGNAL(clicked()), this, SLOT(removeField()));
    layout->addWidget(button);
    buttons_.append(button);

    field->setLayout(layout);

    return field;
}
```

Виджет, который будет содержать в себе все элементы поля
И компоновщик для него

Убираем отступы от краёв виджета, они нам не нужны

При изменении текста в поле ввода
будет меняться и надпись

StretchFactor регулирует пропорции виджетов при изменении
размеров компоновщика (по умолчанию 0).
В нашем случае кнопка будет занимать минимально возможное
пространство (StretchFactor = 0), а надпись и поле ввода будут
делить оставшееся место поровну (StretchFactor = 1)

```
ui->setupUi(this);
```

Конструктор

```
fieldsLayout_ = new QVBoxLayout;
```

```
fieldsLayout_->addStretch();
```

Добавляет QSpacerItem (в нашем случае - VerticalSpacer) в компоновщик

```
ui->scrollAreaWidgetContents->setLayout(fieldsLayout_);
```

```
connect(ui->pushButton_AddField, SIGNAL(clicked()), this, SLOT(addField()));
```

```
}
```

```
DynamicWidget::~DynamicWidget()
```

```
{...}
```

Деструктор

```
void DynamicWidget::removeField(const int &index)
```

```
{
```

```
    if (index < 0 || index >= fieldsLayout_->count()) {
```

```
        return;
```

```
    }
```

```
    buttons_.removeAt(index);
```

```
    delete fieldsLayout_->takeAt(index)->widget();
```

```
}
```

```
//public slots:
```

```
void DynamicWidget::addField()
```

```
{
```

```
    fieldsLayout_->insertWidget(fieldsLayout_->count() - 1, createField());
```

```
}
```

count() - 1, т.к. новое поле нужно вставить перед Spacer'ом

```
void DynamicWidget::removeField()
```

```
{
```

```
    removeField(buttons_.indexOf(sender()));
```

```
}
```

Возвращает QObject * - указатель на объект-отправитель сигнала

DynamicWidget

Add field

текст	текст	Remove field
another text	another text	Remove field
1000101101	1000101101	Remove field



DynamicWidget

Add field

текст	текст	Remove field
1000101101	1000101101	Remove field

DynamicWidget

Add field

текст	текст	Remove field
1000101101	1000101101	Remove field



DynamicWidget

Add field

текст	текст	Remove field
		Remove field

Простите, а за чей счёт весь этот банкет?
В смысле, кто память чистить будет?

В Qt для виджета всегда можно указать родительский виджет. При удалении родительского виджета происходит автоматическое удаление всех дочерних виджетов.